Interactive Terminal-Based Word Search Puzzle Generator in C

Course Code & Section: CSE115.4
Project Group No.: Group 10
Date of Submission: August 2026

Group Members

- **Md. Fahad Kamal Lingkon** (ID: 2621146042) — *Grid Generation, Algorithmic Engine & File I/O*
- **Md. Farhad Prodhan Fahim** (ID: 2624625642) — *UI/UX, Console Rendering & State Structures*
- **Mahiraj Promo** (ID: 2622117042) — *Search Engine, Menus & Input Validation*
- **Md. Mahedi Hassan Sumonto** (ID: 2624122042) — *Quality Assurance, Testing & Bug Fixing*

1 Group Contribution Table

Table 1: Module Allocation and Contribution Breakdown

Member Name	Student ID	Assigned Modules / Responsibilities	Contribution
Md. Fahad Kamal Lingkon	2621146042	**Grid Generation, Algorithms & File I/O:** Implemented vector mathematics (DR, DC), collision detection (`canPlace`), Monte Carlo randomized placement, and external file parsing (`loadWordBankFromFile`).	25.0%
Md. Farhad Prodhan Fahim	2624625642	**UI/UX, Console Rendering & State Structures:** Developed Win32 terminal color mapping (`PAL`), dynamic board rendering, checklist formatting, and designed the `PlayerProfile` struct for session tracking.	25.0%
Mahiraj Promo	2622117042	**Search Engine, Menus & Input Validation:** Built the 4-level nested matrix scan (`findInGrid`), hint candidate pool stack, string sanitization pipelines, and interactive game state loops.	25.0%
Md. Mahedi Hassan Sumonto	2624122042	**Quality Assurance, Testing & Bug Fixing:** Conducted full system integration testing, memory handling, stream buffer sanitization verification, edge-case debugging, and game loop stabilization.	25.0%

2 Objectives

- **Procedural Matrix Generation:** Design and construct a dynamic 15×15 grid capable of embedding vocabulary strings across all 8 cardinal and diagonal directions without hard-coded pathing.
- **Collision-Resistant Placement:** Implement mathematical vector models that validate grid boundaries and permit interlocking character intersections while rejecting conflicting letter overwrites.
- **Dynamic Resource Loading:** Implement standard File I/O mechanisms to parse external text files (`.txt`) for word banks, preventing hardcoded limitations and enhancing replayability.
- **Persistent State Tracking:** Utilize C structures (`struct`) to encapsulate and track player session data, including usernames, cumulative scores, and total puzzles completed.
- **Exhaustive Matrix Traversal:** Develop a multidimensional search engine to scan and verify user-spotted words in real time.
- **Defensive Input Handling:** Build robust input-sanitization pipelines preventing buffer overflow vulnerabilities, case discrepancies, and stream synchronization issues.

3 Development Tools & Header Files Justification

IDE & Compiler: Code::Blocks IDE using the GNU GCC Compiler (C99/C11 standard).

Required Header Files & Technical Justification

- `<stdio.h>` **(Standard Input/Output):** Essential for stream operations. Used for formatted console rendering (`printf()`), robust stream reading (`fgets()`, `scanf()`), and crucially for File I/O via `FILE` pointers (`fopen()`, `fprintf()`, `fclose()`) to read and generate the external word dictionary.
- `<stdlib.h>` **(Standard Utility Library):** Supplies the pseudo-random generation pipeline (`rand()`, `srand()`) and terminal control commands (`system("cls")`).
- `<string.h>` **(String & Memory Manipulation):** Provides low-level memory clearing (`memset()`), string length validation (`strlen()`), dictionary matching (`strcmp()`), and memory-safe copying (`strncpy()`).
- `<time.h>` **(Time Utilities):** Supplies `time(NULL)` to retrieve the Unix timestamp, seeding `srand()` so every game generates a unique matrix layout.
- `<ctype.h>` **(Character Handling):** Used for string normalization via `toupper()` and custom word sanitization via `isalpha()`.
- `<windows.h>` **(Win32 Console API):** Conditionally compiled (`#ifdef _WIN32`) to acquire terminal output handles and control text attributes for vibrant color highlights.

4 Technical Writing (Logics, Functions & Structures)

4.1 System Architecture & Global State Structures

The game maintains state using parallel 2D matrices and a dedicated C structure for player data:

Listing 1: Global State Matrices and Player Profile Struct

```c
/* Core Grid Matrices */
static char grid[15][15]; // Holds uppercase letters displayed to the user
static int fmask[15][15]; // Mask layer storing color IDs of user-found words
static int smask[15][15]; // Mask layer storing underlying placed word IDs

/* Player Profile Structure */
typedef struct {
    char username[32];
    int totalSessionScore; /* Total words found across all puzzles played */
    int puzzlesCompleted;
} PlayerProfile;

static PlayerProfile player; /* Global instance of the active player */
```

Terminal Output & UI Screenshot Placeholder

Visual representation of the 15×15 Grid Layout, Color-coded Terminal UI, and Player Profile Summary.

Figure 1: Terminal output showing dynamic grid rendering and the struct-driven player stat tracker.

4.2 Algorithmic Logic: Grid Generation & Vector Math

The engine models spatial navigation in 2D space using coordinate offset vectors:

$$DR[8] = \{0, 0, 1, -1, 1, 1, -1, -1\}$$
$$DC[8] = \{1, -1, 0, 0, 1, -1, 1, -1\}$$

For any character i of word w starting at origin (r_0, c_0) in direction d:

$$r = r_0 + i \cdot DR[d], \quad c = c_0 + i \cdot DC[d] \tag{1}$$

- `canPlace()`: Evaluates proposed vector trajectories. It ensures coordinates satisfy $0 \leq r < 15$ and $0 \leq c < 15$. It checks collision states via:

$$\text{if } (\text{grid}[r][c] \neq 0 \text{ and grid}[r][c] \neq w[i]) \implies \text{Reject } (0) \tag{2}$$

- `tryPlaceWord()`: Implements a Monte Carlo trial loop capped at TRIES $= 1000$. It samples random origins and directions, placing the word via `doPlace()` upon validation.
- `fillRandom()`: Fills unassigned cells (0) with pseudo-random uppercase ASCII characters:

$$\text{letter} = \text{'A'} + (\text{rand}() \bmod 26) \tag{3}$$

Source Code Screenshot Placeholder

Implementation of canPlace() validation and tryPlaceWord() Monte Carlo placement loop.

Figure 2: Source implementation of `canPlace()` validation and `tryPlaceWord()` Monte Carlo loop.

4.3 Search Engine & Gameplay Mechanics

Search Engine Code Screenshot Placeholder

Source snippet showcasing findInGrid() 4-nested loop and doGuessWord() state handler.

Figure 3: Search engine implementation featuring 4-nested matrix traversal.

- `findInGrid()`: Executes an exhaustive $O(R \cdot C \cdot D \cdot L)$ search. It evaluates all rows r, columns c, and direction vectors d against target string lengths to locate full or accidental character matches on the board.
- `doGuessWord()`: Validates inputs using `strcmp()`. If valid, it updates `fmask`, increments `player.totalSessionScore`, and checks the win condition:

$$\text{if } (n_{\text{Found}} == n_{\text{Placed}}) \implies \text{player.puzzlesCompleted++ and Trigger Victory} \tag{4}$$

4.4 File I/O Engine: External Word Banks

To allow custom themes without recompiling, the program replaces hardcoded arrays with a dynamic File I/O parser (`loadWordBankFromFile()`).

- **Failsafe Generation:** The program first attempts to open `words.txt` in read mode (`"r"`). If the pointer returns `NULL` (file missing), the program safely enters write mode (`"w"`), generates a default tech dictionary, and automatically re-opens the file for reading.
- **Sanitization:** As strings are pulled via `fgets()`, they are stripped of carriage returns/newlines (`\r\n`), converted to uppercase, checked for duplicates, and filtered by the 14-character `MAX_L` limit.

4.5 Defensive Input Validation & Buffer Sanitization

To prevent crashes from user input errors, the input pipeline utilizes:

- `fgets()` & `strcspn()`: Prevents buffer overflow by restricting reads to buffer capacities and strips trailing newline characters.

- `flushInput()`: Clears residual newline bytes from `stdin` following `scanf()` or interrupted `fgets()` calls to prevent stream pollution.

4.6 Execution Workflow Summary

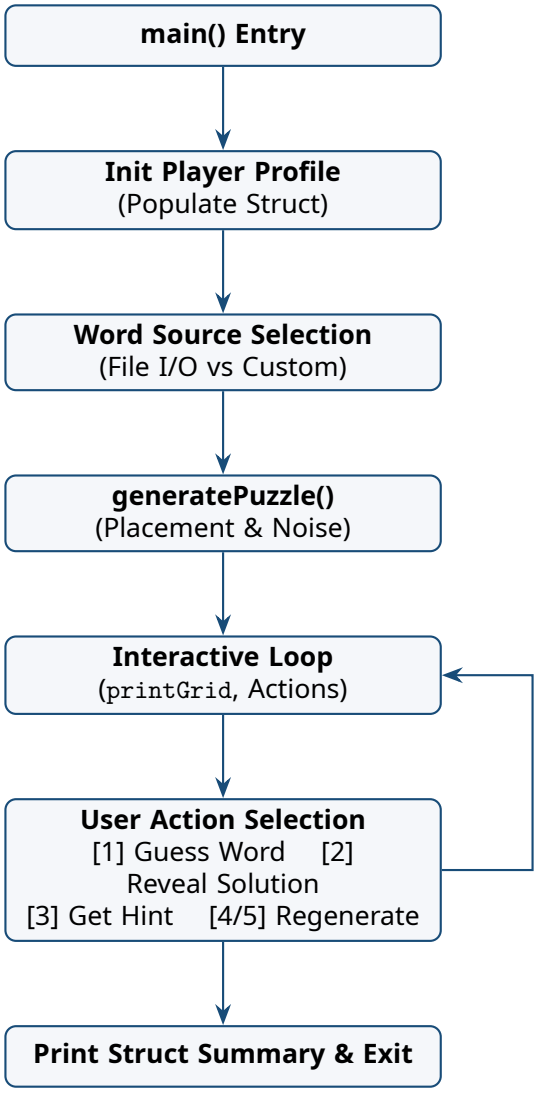

Figure 4: System execution pipeline and interactive state flow.